

```
*Main> :t last
last :: [a] -> a
```

The 'a' in the above snippet refers to a type variable and can represent any type. Thus, the last function can operate on a list of integers or characters (string):

```
*Main> last [1, 2, 3, 4, 5]
5
*Main> last "Hello, World"
'd'
```

You can use a *where* clause for local definitions inside a function, as shown in the following example, to compute the area of a circle:

```
areaOfCircle :: Float -> Float
areaOfCircle radius = pi * radius * radius
  where pi = 3.1415
```

Loading it in GHCi and computing the area for radius 1 gives:

```
*Main> areaOfCircle 1
3.1415
```

You can also use the *let* expression with the *in* statement to compute the area of a circle:

```
areaOfCircle :: Float -> Float
areaOfCircle radius = let pi = 3.1415 in pi * radius * radius
```

Executing the above with input radius 1 gives:

```
*Main> areaOfCircle 1
3.1415
```

Indentation is very important in Haskell as it helps in code readability — the compiler will emit errors otherwise. You must make use of white spaces instead of tab when aligning code. If the *let* and *in* constructs in a function span multiple lines, they must be aligned vertically as shown below:

```
compute :: Integer -> Integer -> Integer
compute x y =
  let a = x + 1
      b = y + 2
  in
    a * b
```

Loading the example with GHCi, you get the following output:

```
*Main> compute 1 2
8
```

Similarly, the *if* and *else* constructs must be neatly aligned. The *else* statement is mandatory in Haskell. For example:

```
sign :: Integer -> String
sign x =
  if x > 0
  then "Positive"
  else
    if x < 0
    then "Negative"
    else "Zero"
```

Running the example with GHCi, you get:

```
*Main> sign 0
"Zero"
*Main> sign 1
"Positive"
*Main> sign (-1)
"Negative"
```

The case construct can be used for pattern matching against possible expression values. It needs to be combined with the *of* keyword. The different values need to be aligned and the resulting action must be specified after the '->' symbol for each case. For example:

```
sign :: Integer -> String
sign x =
  case compare x 0 of
    LT -> "Negative"
    GT -> "Positive"
    EQ -> "Zero"
```

The compare function compares two arguments and returns *LT* if the first argument is lesser than the second, *GT* if the first argument is greater than the second, and *EQ* if both are equal. Executing the above example, you get:

```
*Main> sign 2
"Positive"
*Main> sign 0
"Zero"
*Main> sign (-2)
"Negative"
```

The *sign* function can also be expressed using guards ('|') for readability. The action for a matching case must be specified after the '=' sign. You can use a default guard with the *otherwise* keyword:

```
sign :: Integer -> String
sign x
  | x > 0 = "Positive"
```